

Parquet, Transaction, Delta Table, Upsert

Training / Learning Documentation

Prepared by: **SHINY ZERUBBA RAJIVE S**

Date: 30-Apr-2026

1. What is Parquet?

Parquet is a **file format used to store data**, similar to CSV or Excel, but more efficient.

CSV (row-based storage)

```
id,name,salary
1,Arun,50000
2,Ravi,60000
```

Parquet (column-based storage)

```
id      → [1, 2]
name    → [Arun, Ravi]
salary → [50000, 60000]
```

Why Parquet is useful

- Faster queries (reads only required columns)
- Uses less storage (compression)
- Designed for big data systems like Databricks and Spark

Why Parquet is powerful?

Faster

If you need only salary:

- CSV → reads everything
- Parquet → reads only salary

Less storage: Uses compression

Big data friendly: Used in Databricks, Spark

2. What is a Transaction?

A **transaction** is **one complete operation** that either fully succeeds or fully fails.

Real-life example:

ATM withdrawal:

1. Check balance
2. Deduct money
3. Confirm

All these together = **one transaction**

Key idea

- If something goes wrong in the middle, the system cancels the whole operation.
- This prevents partial updates and data corruption.

3. What is a Delta Table?

A **Delta Table** is a combination of:

- Parquet files (actual data)
- Transaction log (history of changes)

4. Internal Structure

Delta Table Folder

```
|
|— part-001.parquet  (data)
|— part-002.parquet
|
|— _delta_log/      (history)
    |— 000000.json
    |— 000001.json
```

5. What is _delta_log?

The `_delta_log` folder stores **all changes made to the data**.

Each file represents one change.

Example

000000.json

Added: part-001.parquet

000001.json

Removed old file

Added updated file

6. Important Concept

Delta tables do not update files directly.

Instead:

1. Old file is kept as it is
2. New file is created with updated data
3. Change is recorded in `_delta_log`

7. Why this design is important

Time travel

You can access previous versions of data.

Data safety

Even if a system fails, old data is still available.

Concurrent usage

Multiple users can work on the same data safely.

1. What is MERGE (UPSERT)?

👉 MERGE = UPDATE + INSERT (UPSERT)

Think like this:

You already have a table -> retail_orders

New data is coming -> new_orders

Now question:

“If same order already exists → UPDATE”

“If order is new → INSERT”

That’s exactly what **MERGE** does

Real Life Analogy

Imagine:

- You have a **student attendance register**
- New attendance sheet comes

Now you will:

- If student already exists → update attendance
- If new student → add to register

👉 This is MERGE

2. Structure of MERGE

Basic syntax:

```
MERGE INTO target_table
USING source_table
ON matching_condition
```

```
WHEN MATCHED THEN
    UPDATE ...
```

```
WHEN NOT MATCHED THEN
    INSERT ...
```

3. Now Your Case (Step by Step)

Step 1: Target Table

retail_orders

👉 This is your **main table (existing data)**

Step 2: Source Table

new_orders / incoming_orders

👉 This is **new data coming**

Step 3: Matching Condition

ON target.order_id = source.order_id

👉 Meaning:

- Same order_id → same order

4. MERGE Logic (Core Understanding)

CASE 1: WHEN MATCHED

WHEN MATCHED THEN UPDATE

👉 If order_id already exists:

Example: 102 already exists

Old:

(102, 'Priya', quantity=2, status='Placed')

New:

(102, 'Priya', quantity=4, status='Delivered')

👉 After UPDATE:

quantity → 4

status → Delivered

CASE 2: WHEN NOT MATCHED

WHEN NOT MATCHED THEN INSERT

👉 If order_id NOT present:

Example: 109, 110 → not in retail_orders

👉 They will be inserted